

開発管理統合 アプリケーション

日揮パラルテクノロジー株式会社 インターン課題

2026 年

システム概要

解決した課題と全体像

課題	申請システムがバラバラ		解決	申請・承認を一元化
課題	進捗管理が Excel 分散		解決	タスク・進捗を Web 管理
課題	予算追跡がリアルタイムでない		解決	予算・工数を自動集計・即時反映

※ 当初は幅広い機能実装を検討したが、利用者ヒアリングと実務フローを踏まえ「ウォーターフォール型開発案件の予実可視化PoC」へスコープを再整理した。

技術スタック： Vue 3 / TypeScript | FastAPI / Python 3.12 | MySQL 8.0 | Docker / Oracle Cloud

技術スタック

フロントエンド

Vue 3 / TypeScript
Vite / Pinia / Vue Router
Element Plus
@vueuse/core
fetch API

バックエンド

Python 3.12 / FastAPI
SQLAlchemy 2.0 / Alembic
Pydantic v2
JWT 認証 (python-jose)
WebSocket / pytest

DB / インフラ

MySQL 8.0
Docker / Docker Compose
Oracle Cloud (Ubuntu 24.04)

課題 1：実装機能一覧

ロール別機能マトリクス

機能	申請者	担当者 (TASK_MEMBER)	部門管理者	本部管理者
案件申請	✓	—	—	—
案件閲覧	自分のみ	自部門のみ	自部門のみ	全部門
一次承認	—	—	✓	—
最終承認	—	—	—	✓
タスク管理	✓	✓ (タスク更新)	—	—
予算・工数入力	✓	—	—	—
予算一覧閲覧	—	—	自部門のみ	全部門
着手操作	✓	—	✓	✓
完了操作	—	—	✓	✓
アラートダッシュボード	—	—	✓	✓

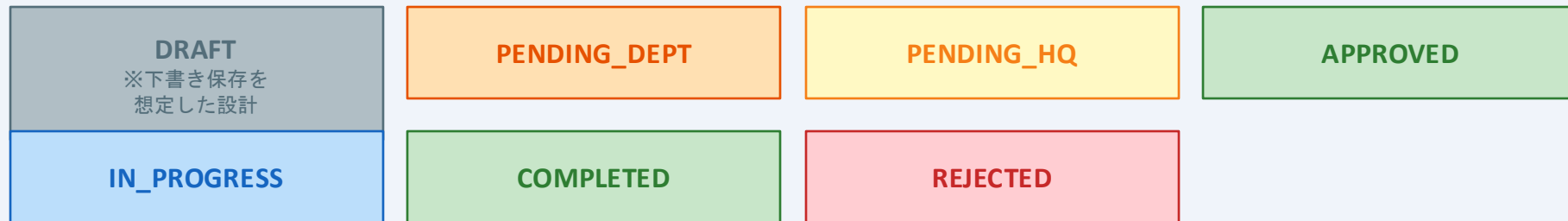
★TASK_MEMBER実務運用設計：担当者不在・休暇・引継ぎを想定し、同部門内で代理更新可能な設計を採用。品質担保のため、機能レビュー / 完了への遷移と進捗100%更新は管理者のみ許可。

課題 1：承認フロー

申請から完了までのステータス遷移



ステータス一覧



課題 1 : 工夫点①

サービス層の責務分離・SQL 最適化

サービス層の責務分離

PermissionService

データアクセス権限チェック

ApprovalService

承認・却下フロー制御

BudgetService

予算サマリ・消化率計算

WorklogService

工数実績の集計

ExpenseService

直接経費の管理

バックエンド集計の SQL 最適化

- N+1 問題を SQL 集計に統一
- 予算サマリをバックエンドで一元計算
- 消化率・加重平均も DB 側で集計
- サーバーサイドページネーション実装
- デバウンス処理付きフィルター

課題 1 : 工夫点②

WebSocket 通知 ・ バリデーション ・ テスト

WebSocket によるリアルタイム通知

- 承認・却下・着手・完了時に即時プッシュ通知
- 自動再接続（最大 10 回）
- 画面右下トースト表示
- 通知UIをPC（Popover） / スマホ（Drawer）に最適化
- 通知種別ごとの色分け表示で優先度を直感的に判断可能

入力バリデーション強化

- 日付整合性チェック（開始日・完了日・期間外タスク）
- 予算・工数・経費の上限値チェック
- 二重送信防止・入力値の自動補正

テストコード（pytest・32件）

- 権限チェック・承認フロー・案件操作
- conftest.py で共通フィクスチャを整備
- 単一責任の原則でテスト・保守しやすい構成

課題 1：セキュリティ・品質向上

評価レポートを受けた改善対応

セキュリティ改善

IDOR脆弱性修正

子リソース（tasks/budget/worklogs/expenses）に親リソースと同様の認可チェックを追加

URL直打ちアクセス制御

フロントのルーターガードとバックエンドAPIの二重チェックで不正アクセスを遮断

Frontend + Backend 二重認可

PermissionServiceを全エンドポイントに統一適用し認可漏れを解消

品質向上

pytest

権限チェック・承認フロー・案件操作の自動テスト（32件）

Ruff

Pythonコード静的解析・フォーマット統一

TypeScript strict

型安全性の強化・暗黙的anyの排除

deploy_check.sh

デプロイ前の自動チェック（env整合性・DB接続・ビルド確認）

「動く」だけでなく「安全に動く」実装の重要性を実感した改善サイクル

課題 2：取り組み方針

設計思想と優先順位の決め方

課題1の基盤をもとに、ヒアリング結果と実際の操作フィードバックから、「現場で本当に使われるシステム」を目指した機能拡張案を設計しました。

フィードバックを通じて、PM・部門管理者・本部管理者で必要情報が異なることを認識。詳細タスク管理より**「案件予実可視化」を重視し、本PoCはウォーターフォール型案件管理**へスコープを整理しました。本格的なアジャイル管理は将来拡張の対象として位置付けています。

優先順位の決定基準

- ① 現場の業務課題に直結するか
- ② 既存データ・API に対応可能か（追加コストが低いか）
- ③ 管理職・現場担当者の双方に価値があるか

課題 2：ウォーターフォール型案件管理

ウォーターフォール型開発案件を対象として設計・実装

背景：開発案件では工程管理・承認フロー・予算進捗管理を段階的に確認する必要がある。本PoCではウォーターフォール型開発案件を対象としてスコープを整理し、設計・実装した。

ウォーターフォール型

- 工程ベースのタスク管理
- 工程列（phase_name）表示
- 「工程→担当者→作業詳細」の流れで確認できる表示
- ガントチャート：工程識別しやすい色分け表示
- 承認後～着手前に開発方針（工程・担当体制）を整理できる設計

課題 2 : ガントチャート

実装済み

実装内容

- 工程進捗の視覚化（工程単位のガント表示・工程別カラー表示）
- 担当者フィルター・日/週/月 表示切り替え
- 本日ラインの表示（赤い縦線）
- 本日周辺を初期表示し、「今日へ移動」ボタンで本日ラインへ即時復帰
- 工程順・担当者順のソート
- 「工程→担当者→作業詳細」の階層構造
- タスククリックで詳細Dialogを表示し、一覧を維持したまま詳細確認可能

設計上の工夫

ウォーターフォール型では
「工程 → 担当者 → 作業詳細」
の流れで確認できる表示にすることで、

「どの工程で止まっているか」
↓
「誰の作業が止まっているか」

という実務の確認フローに沿った設計を採用しました。また、本日周辺を初期表示し、「今日へ移動」ボタンで本日ラインへ戻れるようにすることで、長期間案件でも現在位置を見失わずに確認できるUXを重視しました。

本PoCでは、詳細WBS管理ではなく、案件状況を俯瞰するための概要管理UIとして設計しました。

課題 2 : アラートダッシュボード

部門管理者・本部管理者向けの早期異常検知

アラートレベル判定基準（固定閾値方式）

危険	スケジュール乖離 -20% 以上	予算乖離 -25% 以上	期限超過タスク 2件以上
注意	スケジュール乖離 -10% 以上	予算乖離 -15% 以上	期限超過タスク 1件以上

設計上の工夫：2層構造

Layer 1 表面	Layer 2 詳細
危険 / 注意 のシンプル表示 管理者が一目で異常を把握	SPI（スケジュール効率） CPI（コスト効率）の数値確認
★ ヒアリング起点：本部管理者から「全案件の進捗を一覧で把握したい」という要望をもとに実装。 ★ HQ_MANAGER UX改善：danger → warning → normal の自動ソートにより緊急案件が最上位表示。2列グリッドで一覧視認性を向上。 ★ UI改善：サマリカード選択時に対象案件一覧を同一画面内で即時表示し、危険・注意案件をすばやく確認できる導線を実装。 また、部門タブ選択により対象部門の案件・KPIを即時フィルタ表示できるようにし、多部門環境でも必要情報へ素早く到達できるUXを実現。 ★ 状態管理：KPIカードによる一覧切替はコンポーネント内部状態で管理。ブラウザ更新時は初期ダッシュボード表示へ復帰する設計とした。	

課題 2：メンバータスク状況

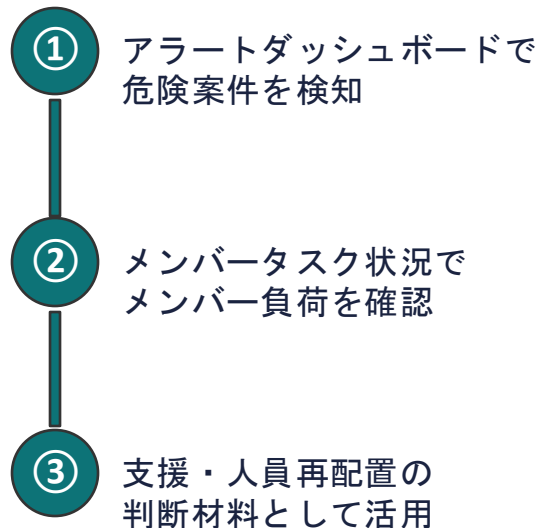
部門管理者向けの人員管理支援機能

実装内容

- ・ メンバーごとのサマリカード表示
- ・ 担当タスク数・進行中・完了・未着手・完了率
- ・ 期限超過メンバーを赤背景で強調
- ・ 担当タスクの進捗更新（未着手・進行中・レビュー中・完了）
- ・ 担当タスク件数からメンバー負荷を可視化

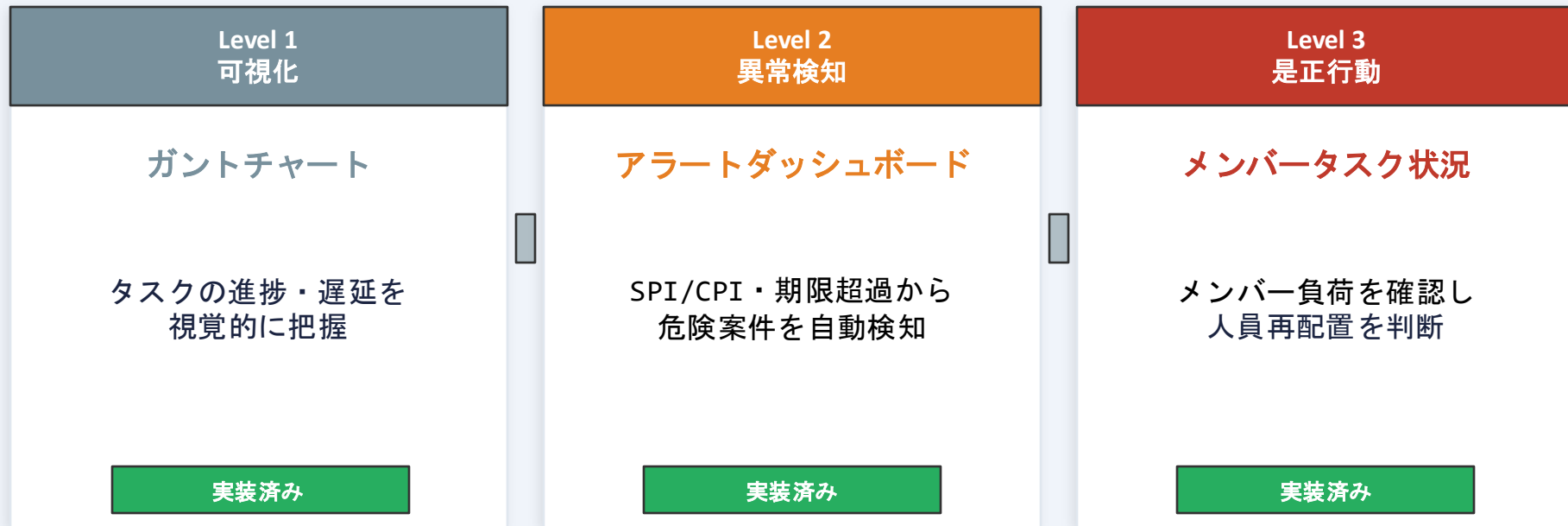
※ 現在は件数ベースでの負荷把握。将来的に工数ベースの稼働率表示へ拡張予定。

活用フロー



課題 2 : 3機能の連携

案件状況可視化 → 異常検知 → 是正判断



設計思想

「意思決定しやすいUI」を目指し、3機能を段階的に設計・実装。

案件可視化 → 異常検知 → 是正判断という実務フローに沿って、

ガントチャート・アラートダッシュボード・メンバータスク状況を連携させたUIを構築しました。

さらに利用者フィードバックをもとに画面遷移を削減し、「一覧確認 → 詳細確認 → 判断 → 操作」までを同一画面内で完結できるUIへ改善した（詳細は次頁）。

実務レビュー・操作フィードバックからの改善提案

実際の操作を通じて浮かび上がった課題と改善案

改善提案① 実績開始日入力

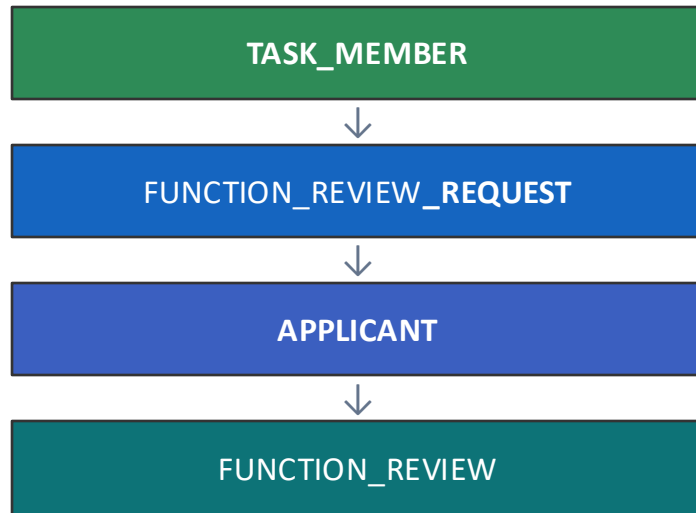
課題

予定開始日のみで実績開始日が記録できない
→ 予定と実績の差異分析ができない

効果

- 予定/実績差異分析
- 遅延の早期検知
- KPIの精度向上

改善提案② 機能レビュー依頼フロー



効果：品質管理フローの可視化・承認プロセスの透明性向上

▶ 操作レビュー・操作フィードバックを通じて、「計画管理だけでなく実績管理の重要性」「ロール別に必要情報が異なること」「詳細管理は既存ツール併用が自然であること」を再認識し、本PoCは「開発案件の申請・承認・予実可視化」にScopeを整理しました。

改善提案③ 画面遷移を抑えたUI改善

実際の操作フィードバックから浮かび上がった課題と対応

背景・課題

操作フィードバックで確認された課題

：

- ・ 戻る操作が多い
- ・ 一覧の絞り込み条件が失われる
- ・ 詳細確認のたびに文脈が途切れる

実装済み

案件申請をDrawer表示へ変更

- ・ ダッシュボードの文脈を維持
- ・ 一覧位置を維持・戻る操作不要
- ・ URL query (/?action=create-project) と状態同期。Drawer閉鎖時はqueryを自動削除。ブラウザ再読み込み時も状態復元可能。

① Dialog表示

案件詳細・承認コメント・通知詳細など短時間確認に適用

- ・ 一覧を表示したまま詳細確認
- ・ 確認後の戻る操作不要
- ・ 承認・却下など判断操作と相性が良い

② Popover表示

通知一覧・補足情報など軽量な確認に適用

- ・ 小さな情報をすぐ確認できる
- ・ 画面全体を切り替えずに済む
- ・ 通知確認の負担を軽減できる
- ・ PC : Popover / スマホ : Drawer で表示切替
- ・ 通知選択時はDialog表示。一覧を維持したまま詳細確認・既読化が可能。

③ 画面内パネル切替

ダッシュボード内の案件一覧・KPI詳細に適用

- ・ ダッシュボードの文脈を維持
- ・ KPIカードから関連一覧へ直接切り替え
- ・ 「今どの情報を見ているか」を把握しやすい

④ 別ウィンドウ表示（将来案）

複数情報を並べて確認したい場合の拡張候補

- ・ 元画面を残したまま別画面で入力
- ・ 複数案件の比較・参照に利用可能
- ・ ※同時編集による保存競合に注意

改善提案③ 他機能への展開例とUI使い分け

画面遷移を抑えた設計を各機能へ展開し、利用者の操作体験を継続改善する

他機能への展開例

案件詳細 → DialogまたはDrawerで表示

- ・一覧の絞り込み条件・表示位置を維持・戻る操作不要

承認・却下操作 → DialogでコメントUI表示

- ・一覧上で情報を確認しながら承認判断が可能

タスク詳細 → ガントチャートからDialog表示（実装済み）

- ・スケジュール全体を見ながら個別タスクを確認

通知詳細 → 一覧からDialog表示

- ・通知一覧を閉じずに複数通知の確認が可能

メンバータスクから案件詳細 → DialogまたはDrawer表示

- ・タスク作業中に案件全体の文脈を確認できる
- ・タスク一覧の絞り込み状態を維持できる

用途別UI使い分け

用途	UI
申請・入力フォーム	Drawer
詳細確認	Dialog
通知など軽量確認	Popover
ダッシュボード一覧切替	画面内パネル
複数画面の比較	将来：別ウィンドウ

画面遷移を減らす主なメリット

- ・作業中の文脈を維持できる
- ・戻る操作・再検索の手間を削減できる
- ・一覧と詳細を行き来する業務に向いている
- ・利用者の操作負担を軽減できる

今後の拡張提案

※ 以下は実務レビューをもとに整理した未実装の拡張候補です。

機能	対象	効果
負荷率（稼働率）表示	部門管理者	件数でなく工数ベースでの負荷把握
アサイン候補表示	部門管理者	負荷の低いメンバー候補の提示
月次進捗レポート自動生成	管理者全般	Excel・メール共有の削減／自動化
メール通知	全ロール	アプリ外への通知対応
★実績開始日入力	全ロール	予定/実績差異分析・遅延早期検知
★機能レビュー依頼フロー	担当者 / 申請者	品質管理フローの可視化
★モバイル実運用検証	全ロール	タップ操作性向上・現場利用シーンへの適合性向上
★ダッシュボードKPI項目最適化	管理者全般	意思決定速度向上・優先KPI表示の継続最適化

追加		
★案件責任者表示（PM/PL）	管理者全般：問い合わせ先の明確化・判断速度向上	★URL状態保持 全ロール：ブラウザ更新時の表示復元
★却下理由の可視化	申請者/承認者：再申請導線の明確化	★共有URLによる状態再現 管理者：情報共有の効率化
★通知の再未読化	全ロール：後回しタスクの見落とし防止	
★申請者向けメンバー状況	申請者：遅延原因分析の迅速化	

今後の拡張提案（続き）

機能	対象	効果
年度・部門予算管理	本部管理者 / 部門管理者	年度予算 → 部門予算 → 案件予算の一元管理
費目別・月次予算管理	全ロール	より詳細なコスト管理
原価管理	全ロール	工数・人件費の詳細な紐づけ
予実差異の可視化	全ロール	予定・実績の差異グラフ
ガントチャートドラッグアサイン	部門管理者	その場でタスク再配置
KPI推移分析・予測表示	本部管理者	SPI/CPI推移・案件完了率・投資対効果分析
★差し戻しフロー	承認者全般	コメント付き再申請・承認履歴の透明化
★HTTPS (SSL/TLS) 対応	全ロール	通信暗号化・認証情報保護・本番運用強化
★中止ステータス (CANCELLED)	全ロール	中止案件の履歴保持・コスト分析
★代理承認設定	部門管理者・本部管理者	承認停滞防止・監査性向上
★先行タスク依存制御	PM・担当者	手戻り防止・クリティカルパス可視化
★受注開発向けシステム拡張	全ロール + 営業部門 + 経理部門	顧客・契約・売上・納品管理／自社→受注開発への統合拡張

インターンの感想

1 / 2

1. インターン全体を通じて感じたこと

案件申請・承認フロー・タスク管理・予算管理・通知・ダッシュボードなど複数機能が連携する構成であり、機能追加や設計変更時の影響範囲を意識しながら開発する難しさを実感しました。

前回の成果物の復習が今回の課題への取り組みを後押しし、要件整理から設計へ落とし込む考え方や設計判断の重要性への理解が深まったと思います。

2. 生成AIを活用した開発について感じたこと

生成AIを活用することで一定規模のシステムを形にできる一方、品質の高いアウトプットにつなげるには自身の知識・経験とレビュー力が不可欠だと強く感じました。

複数の実装案を比較・検討する場面も多くありましたが、最終的に何を採用するかを判断するのは自分自身であり、基礎知識や設計力をさらに深めていく必要性を感じました。

3. セキュリティについて感じたこと

インターン期間中に生成AI関連ツールの情報流出懸念やaxiosのサプライチェーン攻撃の話題に触れ、便利な技術も使い方や管理を誤れば大きなリスクになることを改めて実感しました。

今回の開発でもIDOR脆弱性の修正やロール別アクセス制御の見直しを行い、「動くものを作ること」と「安全に使い続けられる仕組みを設計すること」は同じくらい重要であると学びました。

実際の操作フィードバックを通じて、画面遷移の多さや絞り込み条件の喪失など、利用者が感じる「小さなストレス」が積み重なると操作体験を大きく損なうことを実感しました。さらにseed品質監査を通じ、責任者表示・却下理由可視化・通知再利用性など、運用時の情報不足も改善対象として認識することができ、UIは実装して終わりではなく、使われ続ける中で磨かれるものだとし学ぶことができました。

インターンの感想

2 / 2

4. ユーザー視点・UI/UXについて感じたこと

「言われた通りにシステムを作ること」と「実際に使いやすく価値のあるシステムを作ること」は全く別であることを実感しました。実際に親族や実務経験のある方々のフィードバックを通じて、ダッシュボード構成・画面遷移・通知UIを大幅に見直すきっかけとなり、開発者視点だけでなく、実際に利用するユーザーの立場で操作性や導線を考え、継続的に改善していくことの重要性を学びました。

5. 大規模開発・設計管理について感じたこと

機能が増えるにつれて各機能の関連性や設計変更時の影響範囲の把握が想像以上に難しくなることを実感しました。ステータス設計の変更や権限制御の見直し時には、広範囲の修正が必要となり、実装を進めることだけでなく、設計資料と実装内容を継続的に同期させることが保守性向上に重要だと改めて学びました。

6. 今後に向けて

今回のインターンを通じて、技術面・設計面での成長を実感できた一方で、まだ知識や経験が不足している部分も多いと認識しました。生成AIを活用する時代だからこそ、単にツールを使いこなすだけでなく、課題を整理して最適な設計判断を行う力、コードや提案内容を見極めるレビュー力、利用者視点で課題を捉える姿勢、そしてセキュリティ意識がこれまで以上に重要になると感じています。

また、利用者フィードバックや品質監査をもとに課題を整理し、実装や提案へ反映していく一連の経験を通じて、技術そのものだけでなく、相手の立場を理解しながら課題解決へつなげる力は、エンジニアとしてだけでなく、さまざまな業務や組織の中でも活かせる力だと実感しました。

今後は、今回得た経験や学びを大切にしながら、エンジニアとして技術面だけでなく、課題整理・品質向上・利用者視点での改善提案まで行える人材を目指し、継続的な成長につなげていきたいと思っています。